- CPSC 375: High-Performance Computing

CPSC 375: High-Performance Computing

Spring 2026

- Syllabus
- Schedule
- Upload
- The Cluster Project

Assignment 5

Due 11:59 p.m., March 25, 2026

> **IMPORTANT!** This is an individual (or in pair) assignment. You may discuss broad issues of interpretation, understanding, and general approaches to a solution. However, the development of a specific solution or program code must be your own work. The assignment is expected to be entirely your own, designed and coded by you alone. If you need assistance, please consult your instructor or the TAs. Be sure to read the specific policies outlined in the **Academic Honesty in Computing** section.

Introduction

In this assignment, you will integrate core interprocess communication (IPC) concepts—including message queues, TCP sockets, thread pools, mutexes, and condition variables—to build a concurrent transaction processing system. The project is organized in two phases. In Part 1, you will implement a relational database management system that provides the underlying storage structures, data organization, and disk simulation required to support transaction execution. In Part 2, you will extend this underlying system into a realistic simulation of concurrent transaction processing. The primary goal is to design and implement a system that simulates multiple transactions executing concurrently while preserving correctness, consistency, and overall system integrity through appropriate concurrency control and recovery mechanisms.

Part I: Implementation of Relational Database Management System

You are to implement a general relational database management system (DBMS). Such system should have the facility for creation, deletion, and update of a relation. Implement the *select, project, union, set difference* and *join* relational operators. Basically there are two types of relations: *base* and *derived* relations. A base relation is a relation that is created explicitly through a *create* operation. A derived relation is a relation that is obtained as a result of an algebraic operation such as *select*, *join* and so on. Once it is completed, we should be able to use it similar to a commercially available relational DBMS.

Data Dictionary

In addition to relations, the system maintains a *data dictionary*. The data dictionary contains information about every relation (base or derived) in the database such as its name, and names and domains of attributes. Whenever a relation (base or derived) is created, the information regarding the relation is entered in the data dictionary. In the implementation of algebraic operations, the data dictionary is used for checking the compatibility of relations, for determining the command attributes, etc.

The dictionary itself is organized as two base relations called `catalog` and `columns`. The relation `catalog` contains one tuple for each relation in the database including `catalog` and `columns`. The relation `catalog` has six attributes:

1. *Relname*: Key component and its domain is string.
2. *Kind*: Domain is integer.
3. *Attsize*: Domain is integer.
4. *Keysize*: Domain is integer.
5. *Relsize*: Domain is integer.
6. *Relptr*: Domain is integer.

Suppose `t` is a tuple in `catalog` corresponding to some relation. Then, the Relname component of `t` contains the name of the relation. The Kind component specifies whether the relation is base or derived; 0 denotes a base relation and 1 denotes a derived relation. The Attsize component of `t` gives the number of attributes of the relation. The Keysize component gives the number of key components of the relation. For derived relations assume that the entire tuple is the key. The Relsize component of `t` gives the numbers of tuples contained in the relation. The Relptr component of `t` points to the storage structure of the relation, namely, the block number.

The relation `columns` has one tuple for every attribute of every relation (including `catalog` and `columns`) contained in the database. This relation has four attributes:

1. *Relname*: Key component and the domain is string.
2. *Attname*: Key component and the domain is string.
3. *Attdomain*: Domain is integer.
4. *Attposition*: Domain is integer.

Suppose `t` is a tuple in `columns` corresponding to an attribute of some relation. Then, the Relname component of `t` contains the name of the relation, and the Attname component contains the name of the attribute. The Attdomain component of `t` gives the domain of the attribute; 0 denotes a string and 1 denotes an integer. The Attposition component of `t` gives the position of the attribute in the relation.

These dictionary relations are created by the system at startup time. They are also initialized, that is, entries for the dictionary relations are entered in the dictionary relations. These relations are like any other base relation in the sense that they can be used as operands in algebraic operations and hence should be stored as any base relation. Moreover, `catalog` and `columns` need to be updated accordingly after performing some of the operations described in the following section. In order to ensure the integrity of the database, the following restrictions apply:

- Operations Create, Delete, Insert, Remove, and Update are not permitted on the dictionary relations.
- The name of dictionary relations cannot be used to name another base or derived relation.

Buffer Space

You must observe the following limitation on buffer space. Your main memory buffer can hold a maximum of six secondary storage blocks. A maximum of three blocks to hold the data blocks of the relations, and three blocks to hold the header blocks of the relations. In addition, your buffer will also have space to hold the header blocks for `catalog`, and `column` relation, and a block to hold bitmap, which will be described later. Therefore, your buffer can hold 9 blocks.

The DDL and DML Commands

After performing some initialization, the database system accepts commands for data definition language (DDL) and data manipulation language (DML) from the standard input device in the following format:

- Each command is on a new line, and consists of a command verb followed by a list of arguments.
- Each argument is separated from another argument or verb by one of more blanks.
- A verb is composed of two characters.

The following are the commands accepted by the system:

1. **CREATE**
   `CR r n k`
   <attribute specification>
   Create a base relation named `r` with `n` attributes of which the first `k` attributes form the primary key. The specification of each attribute includes its name, and domain (`I` or `S`). The format is `<name> <domain>`. The specification of each attribute is on a new line.

2. **DELETE**
   `DE r`
   Delete the relation named `r`.

3. **INSERT**
   `IN r n`
   <`n` tuples>
   Insert the given `n` tuples in the base relation named `r`. Each tuple is on a new line. Each attribute value is separated by one or more blanks. Uniqueness of keys must be checked.

4. **REMOVE**
   `RM r n`
   <`n` primary keys>
   Remove from the base relation `r`, `n` tuples specified by their primary keys. Each key is given on a new line. Legality of keys must be checked, that is, a key must denote an existing tuple.

5. **UPDATE**
   `UP r n`
   <`n` tuples>
   Update the base relation `r` by replacing `n` existing tuples by the given `n` tuples. Each tuple is on a new line. Legality of keys must be checked. Each of the `n` existing tuples are identified from the key fields of the given `n` tuples. Therefore, note that you cannot update the key attributes of a tuple in a relation. In case one wants to update the key attributes of a tuple then it is done by first deleting the old tuple and then inserting a tuple with the new key value.

6. **PRINT**
   `PR r`
   Print the relation `r` on the standard output device in the following format. On the first line print the name of the relation. On the next line print the names of attributes as column headings. Print tuples on subsequent lines, beginning each tuple on a new line.

7. **UNION**
   `UN p q r`
   Create a new derived relation `r` which is the union of the relation `p` and `q`. Make sure that the relations are compatible, and duplicates are suppressed. The attributes may not be in the same order in relations `p` and `q`. The resulting relation `r` should have the attribute order of the first relation (in this case `p`). A similar case occurs in DIFFERENCE.

8. **DIFFERENCE**
   `DF p q r`
   Create a new derived relation `r` which is the difference of `p` and `q`, that is, `p - q`. Make sure that `p` and `q` are compatible.

9. **PROJECTION**
   `PJ p q n`
   <`n` attribute names>
   Create a new derived relation `q` which is the projection of the relation `p` on the given `n` attributes. Each attribute name is on a new line. Ensure that duplicates are suppressed.

10. **SELECTION**
    `SL p q n`
    <`n` conditions>
    Create a new derived relation `q` which consists of tuples from `p` that satisfy the conjunction of the given `n` conditions. Each condition is on a new line, and is of the form `<op> <value>` where `<op>` is one of the following comparison operators: `==, !=, >, >=, <, <=`, and `<value>` is either an integer or a string.

11. **NATURAL JOIN**
    `NJ p q r n`
    <`n` attribute names>
    Create a new derived relation `r` which is the natural join of the relation `p` and `q`. Attribute names are used to identify common attributes. The first *x* attributes of `r` will be the *x* attributes of `p`.

Bear in mind that in an operation that produces a new relation as a result such as `SL`, `PJ`, etc., the name of an existing base relation cannot be used to name the resulting relation, that is, a base relation cannot be implicitly deleted.

You may assume that commands are syntactically correct. For instance, only an integer appears when an integer is expected, number of key components given in an `IN, RM`, or `UP` commands agree with the scheme of the relation, etc. However, you must incorporate semantic checks such as those for uniqueness of keys, existence of a tuple with the given key, and so on, wherever necessary.

The Storage Structure

Assume the disk medium is simulated by a Linux file. Consider a maximum of 256 secondary storage blocks to be available. Let each block hold 256 bytes. Assume the following organization:

- The first block holds the bitmap information,
- The second block holds the header for the `catalog` relation, and
- The third block holds the header for the `columns` relation.

The bitmap tells us which secondary storage block is used up and which ones are free. Therefore, we have 256 entries in the bitmap block, one corresponding to each of the secondary storage blocks. If an entry is set to 1 then it is being used and if it is 0 then it is free.

The Storage Structure for a Relation

Both base and derived relations are stored on secondary storage blocks. Every relation (base or derived) has a name which is up to 8 characters long. The name of an attribute is also a string with a maximum of 8 characters. The domain of an attribute can be either an integer or a string. If the domain is a string, the attribute value is a string with a maximum of 8 characters.

Every base relation has a primary key that identifies a unique tuple of the relation. The key may be composed of several components. The storage structure for base relations can include auxiliary structures to speed up access based on primary keys. The system does not maintain any information regarding keys for derived relations, that is, all the attributes together form the key in this case. The storage organization is different for the two kinds of relations.

Base Relation. A base relation is stored in a *hash file*. A hash file has the following structure. It has a header block which holds the hash table, some number of data blocks (assume a maximum of 16 blocks), and some overflow blocks (assume a maximum of 4 blocks). Therefore, a base relation can take up at most 21 blocks and at least 2 blocks.

A data block and an overflow block have the following structure: Each block has 4 slots, and each slot can hold a tuple. Thus, a block can hold 4 tuples. In each slot, we use the first byte to indicate whether the slot contains a tuple, and the remaining 63 bytes are used to hold the tuple. Thus, tuples are restricted to 63 bytes in length.

For a given tuple, the data block for that tuple is determined by a hash function. Since we allow a maximum of 16 data blocks, the hash value (that is, bucket number) for a tuple is in the range 1 to 16. All tuples that hash to the same bucket are stored in the same data block. In case a data block is full, further tuples that hash to that bucket will be stored in the first overflow block that has space available. Note that while performing lookup, in the worst case, we may have to sequentially go through each of the four overflow blocks to find the tuple that is being sought.

A simple hash function can be defined as follows. Suppose attributes $A_1, A_2, \ldots, A_n$ form the key k of a relation. Then

$h(k)$ = (sum of the ASCII code for $A_1, A_2, \ldots, A_n$) mod 16.

Derived Relation. A derived relation is stored in a *heap file*, an unordered sequential file. It has a header block and at most 16 data blocks. The header block contains the disk addresses of the data blocks. A data block has the same format as described above. Data blocks are allocated dynamically as the file grows.

Details of the File System

A description of the file system is given below.

```
#define RECSIZE     63      /* Size of data record in bytes */
#define BLKSIZE    256      /* Size of a disk block in bytes */
#define BLKFAC       4      /* Blocking factor */
#define DISKSIZE   256      /* Total number of disk blocks */
#define BLKMAX      16      /* Maximum number of data blocks in a relation */
#define OVBLKMAX     4      /* Maximum number of overflow blocks in a relation */

/* Buffers for data blocks or overflow blocks */
typedef struct slot {
   char flag;               /* 0 = free; 1 = allocated to a record */
   char tuple[RECSIZE];
} slot_t;
slot_t datablk[BLKFAC];
char bitmapblk[DISKSIZE];   /* Buffer for the bitmap block.
                                0 for bitmapblk[i] means that block i is free
                                1 means it is allocated
                            */

/* In addition, you must declare one data buffer and header buffer for each dictionary
   relation. Declare also variables to hold addresses of disk blocks read into these
   buffers.
*/

/* Depending on your implementation you might add or delete some of the arguments
   specified in the following functions */

/* Disk IO operations */
/* Read 256 bytes of data from the specified disk block into the specified buffer */
void diskread(int blocknum, char *buffer)

/* Write 256 bytes of data from the specified buffer into the specified disk block */
void diskwrite(int blocknum, char *buffer)
```

Primitive Functions

The following set of functions constitute the primitive relation access routines. All database operations are implemented using these primitives. In the following, we have used the prefix letter `db` to avoid confusion with Linux file operations.

```
void dbopen(char *relname, char *headerblk)
```

Reads the header block of the relation specified by `relname` into the specified header buffer; data blocks are not read into memory at this time.

```
void dbcreate(char *relname, char *headerblk)
```

This function is used for creating a relation; this operation initializes the specified header buffer and associates with the given header block.

```
int dbread(char *relname, char *tuple)
```

This function is used for sequential scan on a base or derived relation. It reads the next tuple from the data buffer, reading the next block into the buffer, if necessary. Return zero if the end of relation is reached; otherwise return a non-zero value. To read a base relation sequentially, scan data blocks first, and then the overflow blocks. Ensure that unused slots are bypassed while scanning.

```
int dbwrite(char *relname, char *tuple)
```

This function is used for sequential write on a derived relation. It writes the given tuple as the next tuple in the data buffer acquiring a new block if necessary; flush buffer when full.

▣ The following four functions are used for direct access on a base relation using its primary key:

```
int dbget(char *relname, char *tuple)
```

This function retrieves the tuple with the given key, computes the hash value using the key components of the tuple, locate the block, and search for the tuple. If found, return the tuple and the slot number; if the block is full, read overflow blocks and search. Return zero if no matching tuple was found.

```
void dbput(char *relname, char *tuple)
```

This function computes the hash value of the tuple. If a block exists for the hash value, read the block into the data buffer. If the block is full, read an overflow block into the data buffer. If no block exists for that hash value, create a block and initialize the data buffer. Finally, it inserts the given tuple into a free slot in the data buffer and flushes the buffer.

```
void dbupdate(char *relname, int slotnum, char *tuple)
```

This function assumes that the appropriate block is already read into the buffer using `dbget()`. It updates the tuple stored in the specified slot and flushes the data buffer.

```
void dbremove(char *relname, int slotnum)
```

This function assumes that the appropriate block is already read into the buffer using `dbget()`. It deletes the tuple stored in the specified slot and flushes the data buffer.

▣ The following operations are applicable for all relations:

```
void dbclose(char *relname)
```

This function flushes the data buffer if it contains any tuple. It always flushes the header buffer for the relation.

```
void dbdelete(char *headerblk)
```

This function is used to delete a relation, reads the header block, and deletes each data block, overflow blocks, and header block.

Programming Notes

- Write a C program (`mydb.c`) which implements the DBMS as described in the assignment, including:
  - Base and derived relations
  - Data dictionary (catalog and columns)
  - All DDL and DML commands (CR, DE, IN, RM, UP, PR, PJ, SL, UN, DF, NJ)
  - Disk/block simulation and buffer management
- Compile the program with:

```
$ gcc -Wall -o mydb mydb.c
```

- Run the program with:

  ```
  $ ./mydb < mydb.in
  ```

  where `mydb.in` is the input file which contains the commands and data for the simulation. Use the format described in the assignment.

- Sample input: mydb.in
- Output: mydb.out (You do not need to print the comments.)

Handin

When completed, submit your C source, `mydb.c`, on the course website.

References

- *Database System Concepts, Seventh Editio*n by Avi Silberschatz, Henry F. Korth, S. Sudarshan. McGraw-Hill, 2019.
- *An Introduction to Database Systems, 8th Edition* by C. J. Date. Pearson/Addison-Wesley, 2004.

- **Welcome: Sean**
  - LogOut